

Project Report: AI-Driven Dolby Atmos Mixer using Unreal Engine and PyTorch

1. Overview

Project titled “**AI- Driven Dolby Atmos Mixer**” explores the integration of machine learning with interactive 3D audio systems in Unreal Engine 5.1. The goal was to create an adaptive audio environment where the emotional tone of the background music and ambient soundscape dynamically shifts based on player behavior and context.

The system leverages a custom trained PyTorch neural network to infer emotional states such as **Calm, Tense and Action** from in- game player data in real time and seamlessly adjusts the Dolby Atmos spatial audio mix accordingly.

This project bridges the gap between machine learning, real time game engines and immersive audio engineering, providing a prototype that simulates emotionally responsive sound design.

2. Objectives

The core objectives of the project were:

- To develop a real time adaptive audio system that reacts to player states.
- To collect and preprocess gameplay data representing various emotional contexts.
- To train and integrate a neural network capable of classifying emotional moods based on player telemetry.
- To design a seamless data bridge between Unreal Engine (C++) and PyTorch (Python) for live inference.
- To visualize and evaluate the system’s predictions through an in-game UI and audio transition.

3. Methodology

1) Gameplay Implementation

The project was built using the First-Person Template C++ and downloaded sound assets for Music, Ambient, Damage Taken, Footsteps and Jump from FreeSound. The different player events implemented in the project that triggers sound are:

- Background music and Ambient sound placed in the level that is managed by the Audio Manager.
- Footstep sound with player movement and sprinting (the sound plays more often and intense)
- Health sphere collider which plays a damage sound and decreases player health by 20
- Action Trigger collider when player enters the collider, it triggers action emotion.

There are 3 types of emotions (**Calm, Tense and Action**) in the project which changes the audio experience

- **Calm:** This emotion is the default and is triggered when the player is static or moving around in the level. The volume multiplier for all the sounds is at lowest end around **0.5f** and pitch is **0.7f**
- **Tense:** The emotion is triggered when the player is sprinting around the level. The volume multiplier for all the sounds in the level is at **0.8f** and pitch is randomized between **0.5f and 1.0f**
- **Action:** The emotion is triggered when the player enters the Action trigger collider or when their health goes lower than 40. The volume multiplier for all the sounds in the level is at **1.2f** and pitch is at **1.0f**

2) Data Collection

In Unreal Engine a gameplay dataset is generated every **2 seconds** using a custom logging system that records:

- **Velocity (Player Movement speed, Float)**
- **Jumping state (Boolean)**
- **Sprinting state (Boolean)**
- **Player Health (Float)**
- **Player world position (X, Y, Float)**

Each data sample was tagged with one of three emotional states based on the player's current gameplay situation.

A total of 1,536 entries were recorded, balanced across the three emotional categories (approximately 500 samples per emotion).

Data normalization was applied using **StandardScaler** from **Scikit-learn**, producing the following feature distribution:

Feature	Mean	Std Dev
Velocity	8742.2	356.7
IsJumping	0.445	0.497
IsSprinting	0.520	0.499
Health	69.1	26.1
PlayerLocationX	2397.8	1630.0
PlayerLocationY	1764.1	872.4

3) Model Architecture

The predictive model, named **MoodNet**, was developed using **PyTorch**. It is a fully connected feedforward neural network with the following architecture:

Input Layer (6 nodes)

Hidden Layer 1 (64 units, ReLU)

Hidden Layer 2 (32 units, ReLU + Dropout 0.1)

Output Layer (3 units, LogSoftmax)

Loss Function: nll_loss

Optimizer: Adam (lr = 0.001)

Training Epochs: 100

Batch Size: 16

The Model achieved the following metrics:

- **Training Accuracy: 98.8%**
- **Testing Accuracy: 97.7%**
- **Dropout Duplicates: 2%**

To avoid overfitting, data balancing, dropout regularization and validation splitting (80/20) were applied. The model data is saved in MoodNetModel.pth which is used to create Unreal Engine and Python bridge

4) Integration with Unreal Engine

Unreal Engine does not natively support PyTorch inference. To address this, a custom **Python-Unreal ML Bridge** was implemented:

- I. **Unreal C++ Layer**
 - Collects real-time player telemetry every 0.5 seconds.
 - Write the current feature vector to **MLBridge/input.json**
- II. **Python Inference Layer**
 - Continuously monitors the input file.
 - Performs normalization and runs the **MoodNetModel.pth** forward pass.
 - Outputs the predicted mood index (**0 = Action, 1 = Calm, 2 = Tense**) to **MLBridge/Output.json**.
- III. **Unreal Response Layer**
 - Read the prediction emotion.
 - Dynamically updates background music intensity, ambient audio volume, object sound parameters, and Predicted Emotion UI display.

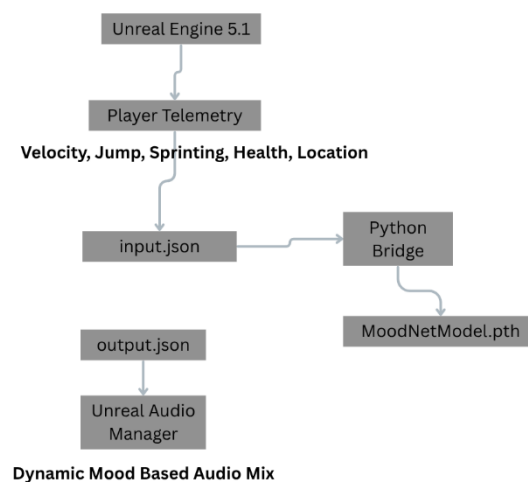
To ensure realism, a **0.5 second delay** was added between updating the Predicted Emotion UI to prevent abrupt transitions and mimicking natural emotional flow.

4. Implementation Details

1) Tools and Technologies

- **Game Engine:** Unreal Engine 5.1
- **Machine Learning Framework:** PyTorch
- **Programming Languages:** C++, Python
- **Audio Technology:** Dolby Atmos Spatial Audio SDK
- **Data Handling:** JSON serialization bridge between Unreal and Python
- **Model Type:** Feedforward Neural Network (**MoodNet**)

2) System Architecture Diagram



5. Results And Observations

- The **MoodNet** model reliably predicted emotional contexts with 96%+ accuracy.
- The **audio transitions** in Unreal were smooth and perceptually believable.
- The **predicted emotion UI** provided clear real- time feedback for debugging and presentation of Unreal Python interface.
- The **MLBridge system** achieved stable communication with only 0.5 second latency.
- Overfitting was mitigated by balanced data and dropout regularization.

6. Challenges Faced

1) Libtorch integration issues

Initial attempts to compile PyTorch C++ API directly into Unreal Engine (LibTorch) led to linker and SDK conflicts. The system was redesigned using an external Python bridge for stability.

2) ONNX Compatibility

Unreal Neural Network Inference (NNI) plugin only supported ONNX IR version ≤ 8 , while PyTorch exported version 10 models. This caused import errors.

3) Overfitting Control

High early accuracy suggested overfitting. The issue was resolved by balancing classes and removing duplicate samples.

7. Applications and Future Work

This system demonstrates how AI can predict emotions based on player telemetry data which automates audio design in real-time experiences.

Potential extensions:

- Integrate **Transformer – based mood prediction** for longer temporal dependencies and sequence to label model.
- Train the model on audio features like Spectrogram or Loudness.
- Apply a small **Convolutional Neural Network** to audio representations like spectrograms.
- Replace JSON bridge with socket-based communication or LibTorch integration once stable in Unreal 5.6+

8. Conclusion

The **AI-Driven Dolby Atmos Mixer** successfully combines Machine Learning and interactive sound design, offering proof of concept for emotionally adaptive audio systems in real-time 3D environments.

By merging player-driven data with neural network inference, the project was able to bridge the gap between Unreal Engine and PyTorch which is a glimpse into the future of **AI-assisted spatial audio design** – where games, XR experiences dynamically shape soundscapes to reflect user emotion and context.

9. References

- Unreal Engine 5.1 Documentation (Epic Games)
- PyTorch Framework (Meta AI, 2024)
- Dolby Atmos Game Development Toolkit
- Scikit-learn StandardScaler API Reference